

Van Reservation System — Architecture

Internal web application for reserving and managing company van transportation at Carelton Global Solutions PH, replacing a temporary Nintex-based process. Associates book rides; OPS Support admins manage the master list, calendar, drivers, and reports.

This document describes how the system is put together: the technology stack, the layering rules, and the main flows (authentication, data access, the request lifecycle). For how to run it, see [README.md](#).

1 At a glance

Framework	Next.js 16 (App Router, Turbopack, React Compiler)
Language	TypeScript (strict), React 19
Database	PostgreSQL 17, accessed through Kysely (typed query builder)
Auth	jose HS256 JWT in an httpOnly cookie; pluggable identity provider
Email	Amazon SES (SESV2), pluggable transport; React Email templates
Styling	Tailwind CSS 4 (@theme tokens), shadcn on Base UI, motion
Validation	Zod (env, session payloads, request bodies)
Tooling	pnpm · Biome (lint/format) · Vitest (unit + integration) · Docker
Size	~11 pages · 4 API routes · ~41 components · 36 unit + 6 integration test files

2 Technology stack

Runtime & framework

- **Next.js** 16.2.12 — App Router, route handlers for the API, edge middleware for the auth gate. Built and served with **Turbopack**.

- **React 19.2** with the **React Compiler** enabled (`next.config.ts: reactCompiler: true, babel-plugin-react-compiler`).
- **TypeScript 5** — path alias `@/* → src/*`.

Data

- **Kysely 0.29** over `pg` — a typed SQL query builder, not an ORM. Table types are **generated** from the live schema by `kysely-codegen` into `src/modules/db/types.d.ts`.
- **kysely-ctl** drives migrations and seeds (`pnpm db:migrate, db:seed`).
- **PostgreSQL 17** in Docker (`docker-compose.yml`, host port 5433).

Auth & security

- **jose 6** — signs/verifies the session JWT (HS256).
- **Zod 4** — the single source of truth for the environment schema, the session payload shape, and request-body parsing.

Email

- **@aws-sdk/client-sesv2** — sends through Amazon SES (v2).
- **React Email** (`@react-email/components + @react-email/render`) — email bodies authored as components, rendered to HTML + plain text.

Frontend

- **Tailwind CSS 4** — configured entirely in `src/app/globals.css` via `@theme` (design tokens) and `@utility` (composed decorative classes). No `tailwind.config.js`.
- **shadcn 4** components built on `@base-ui/react`, themed through a semantic token layer; `class-variance-authority`, `clsx`, `tailwind-merge` (`cn()` helper), `lucide-react` icons, `tw-animate-css`.
- **motion 12** — the van's drive-in on the login screen and the hero "drive-by" swap; imperative `animate()` and declarative `motion/react`.
- **@tanstack/react-query 5** — client-side data fetching/mutations (e.g. the login form's verify + sign-in mutations).
- **sonner** — toasts.

Tooling

- **pnpm** package manager · **Biome** 2.2 (lint + format, replaces ESLint/Prettier) · **Vitest** 4 with `@testing-library/react + jsdom`.

3 Layering & dependency direction

The codebase is split into four layers with a **single downward dependency direction**. Nothing lower ever imports something higher.

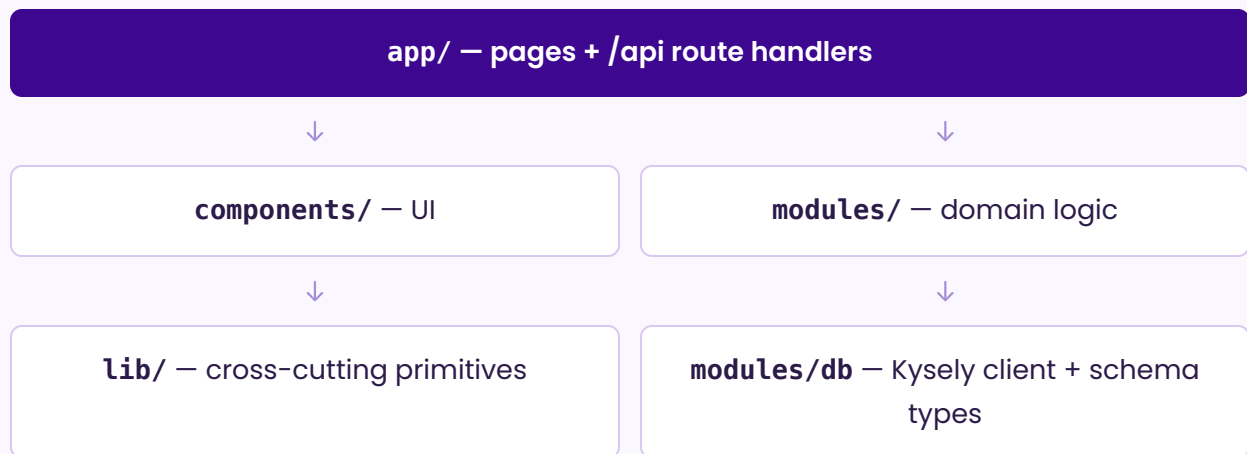


Figure 1. Dependency direction. `app/` depends on `modules/`, `lib/` and `components/`; `components/` depends on `lib/`; `modules/` depends on `lib/` and `modules/db`.

- **src/app** — routes (React Server Components) and `/api` route handlers. The only layer allowed to touch `next/*` request APIs freely.
- **src/modules** — the domain. Subdivided by responsibility (see §4): **pure rules** (no I/O, no framework), **repositories** (SQL), and **services** (orchestration).
- **src/components** — presentation. Client and server components; no business rules.
- **src/lib** — framework-agnostic primitives (`result`, `api-error`, `env`, `tz`, `utils`, `query client`). Held to a **no-next/*** rule so they stay unit-testable without a runtime.

The "pure rules" convention (within modules/)

KIND	EXAMPLE	I/O?	NEXT/*?
Pure rules	auth/roles.ts, auth/roster.ts, reservations/decision.ts	No — take data as arguments	No — unit-tested with no DB
Repository	auth/repo.ts	SQL via modules/db	No
Service	auth/service.ts	Orchestrates rules + repo + provider	authenticate() is injectable & framework-free; getSessionUser() deliberately reads next/headers

This is what keeps the test suite fast: role resolution, roster matching, and reservation rules are exercised as pure functions over fixtures, and only the 6 *.int.test.ts files need a real Postgres.

4 Directory structure

```
src/
├─ app/                                # App Router
│  ├─ (auth)/                          # route group – unauthenticated shells
│  │  ├─ login/ · admin/login/         # the two login screens
│  │  └─ not-authorized/
│  ├─ (requestor)/                    # associate-facing: / , book/ , manage/
│  ├─ (admin)/                        # admin: dashboard, master-list, calendar, reports, drivers
│  ├─ api/auth/                       # login · logout · session · verify-domain (route handlers)
│  └─ layout.tsx · providers.tsx · globals.css
├─ components/
│  ├─ ui/                             # shadcn / Base UI primitives (Button, ...)
│  ├─ auth/                           # login composition (form, hero, van, panel, fields)
│  ├─ requestor/                      # landing hero, booking wizard, manage list
│  ├─ admin/                          # calendar, dashboard, master-list, reports, drivers, trip drawer
│  └─ shells/ · common/               # app shells; shared chips/menus
├─ modules/                           # domain logic
│  ├─ auth/                           # provider seam, service, roles, roster, session, repo
│  ├─ db/                             # Kysely client, generated types, migrations/, seeds/
│  ├─ reservations/                   # ride/status rules, filters, calendar, export, drafts
│  ├─ drivers/                       # driver shifts, weekly-hours workload
│  └─ email/                         # SES transport seam (ses|stub), templates, send service
├─ lib/                              # env, result, api-error, api-fetcher, query-client, tz, utils
├─ hooks/                            # use-hero-swap, use-dismissable
└─ middleware.ts                     # edge auth gate
```

Route groups ((auth), (requestor), (admin)) partition the URL space by audience without adding path segments; each carries its own layout/shell.

5 Authentication & authorization

Auth is the most developed subsystem. Its central design idea is a **provider seam**: everything downstream of identity (roster check → role → session) depends on an interface, never a concrete identity service.

5.1 The provider seam

`modules/auth/provider.ts` defines the only surface an identity service must satisfy:

```
interface AuthProvider {
  identify(req: Request): Promise<Result<AuthIdentity, AuthFailure>>;
  verifyDomain(domainId: string): Promise<Result<DomainCheck, AuthFailure>>;
}
```

Two implementations, chosen by AUTH_MODE in select-provider.ts:

- **StubAuthProvider** (AUTH_MODE=stub) — development only. Resolves a Domain ID against hardcoded fixtures (dev-identities.ts); verifies no password. Refuses to construct outside development/test.
- **CgsAuthProvider** (AUTH_MODE=cgsauth) — the real **CGS Associate Authentication API**. identify() makes two server-to-server calls: POST /api/login (password check → a trimmed profile) then GET /api/associate (enriches with the corporate email, since users.email is NOT NULL). verifyDomain() backs the login form's two-step reveal. Registration endpoints are never called.

Failures are a typed subset of ErrorCode so the UI can tell them apart: NOT_AUTHENTICATED (bad credential), SERVICE_UNAVAILABLE (upstream down / timeout / our key rejected), RATE_LIMITED (upstream cap hit) — three different messages, never all collapsed to "wrong password".

5.2 Sign-in flow

Browser → /api/auth/login	POST { domainId, password }
route → service	authenticate({ db, provider, roster }, req)
service → provider	identify(req) — verify credential (+ enrich) → Result<AuthIdentity>
service	roster.isAuthorized() — FR-2 allowlist, fail-closed
service → Postgres	listActiveWhitelist() → resolveRole() — associate admin_support
service → Postgres	upsertUser(identity, role) — single ON CONFLICT stmt
route → Browser	Set-Cookie vr_session (JWT) + { user, redirectTo }

Figure 2. Sign-in sequence, in order.

`service.authenticate()` runs FR-1→FR-5 in order and is **dependency-injected** (`{ db, provider, roster }`), so it is unit-tested with fakes and never imports `next/*`. The roster check precedes the upsert, so a denied user is never persisted.

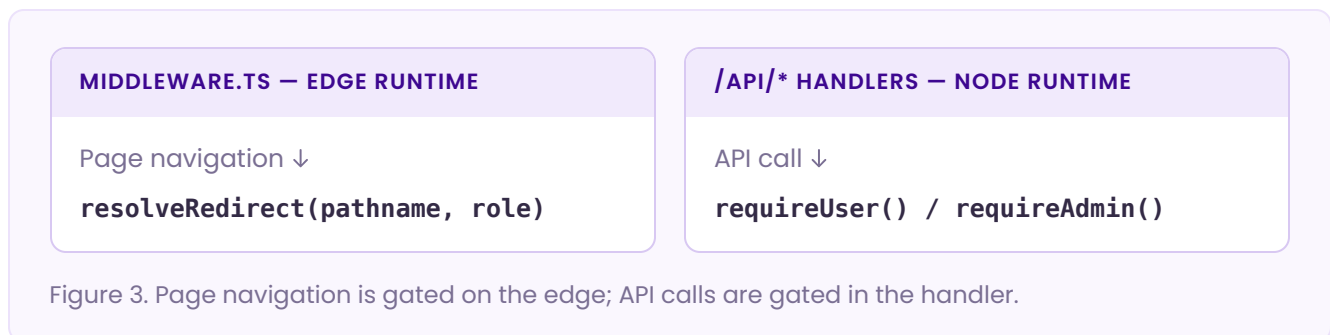
5.3 Roster (who may sign in) & roles (what they may do)

- **Roster** — `AllowlistRoster` matches the identity's Domain ID / email against `ROSTER_ALLOWLIST`. **Fail-closed**: an empty allowlist denies everyone unless `ROSTER_ALLOW_ALL_WHEN_EMPTY=true` is set as an explicit, auditable opt-in (never derived from `NODE_ENV`).
- **Roles** — `resolveRole()` matches against the `admin_whitelist` table; a match → `admin_support`, otherwise `associate`. The match is **allowlist-shaped** and Domain ID wins over email (a Domain ID can't be reassigned).

5.4 Sessions

- `session.ts` signs a **HS256 JWT** (jose) carrying `{ userId, domainId, name, email, role }`, TTL **8 hours**, stored in the **httpOnly vr_session** cookie (`sameSite=lax`, secure in production).
- On read, the payload is re-validated with Zod and `exp` is a **required claim** — a token missing expiry is rejected, not treated as eternal.

5.5 Two enforcement points



- `middleware.ts` gates **page navigation** on the edge. It reads and verifies the JWT **directly** (`readSessionToken`) — it must not import the service, because pulling `getSessionUser()` drags `next/headers` and the `Kysely/pg` runtime into the edge bundle, which fails at request time. `resolveRedirect()` is a pure function (separately unit-tested) that sends anonymous users to the right login, bounces logged-in users off login pages, and blocks non-admins from `/dashboard`, `/master-list`, `/calendar`, `/reports`, `/drivers`.

- **Route handlers** are **excluded** from the middleware matcher (the edge can't load Kysely), so each one calls `requireUser()` or `requireAdmin()` itself — Result-returning primitives in `service.ts`. `requireAdmin()` admits only the literal "admin_support" and denies any future role by default.

Known trade-off: roles live in the JWT and are not re-checked per request, so revoking `admin_support` takes effect on next login or within 8h. Immediate revocation would need a session table or short TTL + refresh.

6 Data layer

6.1 Client

`modules/db/client.ts` builds a single cached `Kysely<DB>` over a pg pool (max: 10) from `DATABASE_URL`. DB is the **generated** type from `kysely-codegen` — schema drift shows up as a type error.

6.2 Schema

users	admin_whitelist
uuid id PK	uuid id PK
text domain_id UK	text full_name
text name	text email unique(lower(email))
text email	text domain_id unique(domain_id)
text mobile_number	text site check: iloilo manila all
text role check: associate admin_support	boolean notify
timestampz last_login_at	boolean active
timestampz created_at	timestampz created_at
timestampz updated_at	

Figure 4. The two auth-side tables.

- **users** — one row per person who has ever signed in; written by `upsertUser()` via a single `INSERT ... ON CONFLICT (domain_id) DO UPDATE`, so a first login can't race itself into two rows. `role` is rewritten on every login.

- **admin_whitelist** — the admin roster consulted by `resolveRole()`. The identity CHECK requires at least one of `email/domain_id`; emails are unique case-insensitively, Domain IDs verbatim (mirrored by `roles.ts`).

Migrations live in `modules/db/migrations/` and run via `kysely-ctl`; the initial seed (`modules/db/seeds/`) loads the admin whitelist. Runtime constants (`APP_ROLES`, `ADMIN_SITES`) are runtime arrays the types derive from, so `schema-constraints.int.test.ts` can assert the DB CHECKs admit exactly the same set.

6.3 Reservations & drivers domain

`modules/reservations/` and `modules/drivers/` hold the **rules** for the booking domain — ride modes (`pickup/standby`), status machine and editable transitions, admin/manage filters, calendar assembly, CSV export, draft handling, driver shifts and the 40-hour weekly cap — as pure functions plus fixtures. The admin and requestor UIs render against these; wiring them to live tables is later-slice work.

7 API & error handling

Route handlers under `app/api/` follow one shape:

1. Read config via `env()`; select provider/roster.
2. Parse the body with `Zod`.
3. Delegate to a service, which returns **Result<T, ErrorCode>**.
4. On failure, `errorResponse(code, message)` emits a uniform envelope.

```
// lib/result.ts — the discriminated union used everywhere instead of throwing
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };

// lib/api-error.ts — codes mapped to HTTP status, one place
{ VALIDATION_FAILED: 422, NOT_AUTHENTICATED: 401, NOT_ON_ROSTER: 403,
  FORBIDDEN: 403, RATE_LIMITED: 429, SERVICE_UNAVAILABLE: 503, ... }

// wire shape
{ "error": { "code": "NOT_ON_ROSTER", "message": "...", "details"?: ... } }
```

On the client, `apiFetch<T>()` (`lib/api-fetcher.ts`) sets JSON headers, parses the envelope, and throws a typed **ApiError** carrying code/status — which is exactly what the login form switches on (e.g. routing `NOT_ON_ROSTER` to `/not-authorized` instead of a toast).

Auth endpoints: `login` (authenticate + set cookie), `logout` (clear cookie), `session` (current user), `verify-domain` (pre-login existence check for the two-step reveal — returns only `{ exists }` to limit enumeration).

8 Frontend & design system

8.1 Rendering

Server Components by default; "use client" only where interactivity requires it (login form, animated van, hero swap). Route groups each get a shell; the browser TanStack Query client is provided in `app/providers.tsx`.

8.2 Design tokens

All theme values live in `src/app/globals.css`:

- **@theme** — the brand palette, requestor/admin login accents, hero gradient stops, gray ramp, type scale, radii, and the shadcn semantic layer (`--color-primary-foreground`, `border-input`, ...) mapped onto brand tokens.
- **@utility** — composed decorative classes that resolve through tokens: `hero-field/login-hero-field` (radial gradients), `*-lobes` (blob highlights), `hero-van-mask/login-van-mask` (edge fades).

8.3 Variant theming

`components/auth/login-theme.ts` centralizes the requestor (purple) vs admin (green) styling as **complete literal class strings** (never string-built, so Tailwind's scanner keeps them). One component set is parameterized by variant rather than duplicated.

8.4 Login composition (worked example)

The login screen is the fullest expression of the system: a clipped diagonal brand wedge (`login-hero.tsx`) with the van breaking past its edge and driving in right→left on load (`login-van.tsx`, motion + reduced-motion aware), the right-panel copy (`login-panel-copy.tsx`), and the two-step form (`login-form.tsx`) — Domain ID (`domain-id-input.tsx`, a 7-box OTP-style input with an n/7 counter) → verify → the password field (`password-field.tsx`) animates up → sign in.

9 Configuration

`lib/env.ts` parses `process.env` **once** through a Zod schema and caches it. It is **fail-closed** by design: `AUTH_MODE` has no default (choosing a provider is an affirmative act), and a `superRefine` refuses to boot when `AUTH_MODE=cgsauth` without `CGS_AUTH_API_KEY`, or when `EMAIL_MODE=ses` without SES credentials. `middleware.ts` re-checks `SESSION_SECRET`'s 32-char floor independently, because the edge runtime never loads `env()`.

VARIABLE	PURPOSE
<code>DATABASE_URL</code> / <code>TEST_DATABASE_URL</code>	Postgres connection (app / integration tests)
<code>SESSION_SECRET</code>	JWT signing key, ≥ 32 chars
<code>AUTH_MODE</code>	<code>stub</code> <code>cgsauth</code> — required, no default
<code>CGS_AUTH_BASE_URL</code> / <code>CGS_AUTH_API_KEY</code>	CGS API base (defaults to prod) + server-only key
<code>ROSTER_ALLOWLIST</code>	Comma-separated Domain IDs / emails permitted to sign in
<code>ROSTER_ALLOW_ALL_WHEN_EMPTY</code>	Explicit dev opt-in; must be unset/ <code>false</code> in prod
<code>APP_URL</code> , <code>NODE_ENV</code>	App origin; environment
<code>EMAIL_MODE</code>	<code>stub</code> <code>ses</code> — defaults to <code>stub</code> (refuses to run in prod)
<code>SES_REGION</code> · <code>SES_ACCESS_KEY_ID</code> · <code>SES_SECRET_ACCESS_KEY</code> · <code>EMAIL_FROM</code>	required-when-ses
<code>EMAIL_REPLY_TO</code>	Optional Reply-To for outbound mail

`.env.example` deliberately ships unsafe values in their **failing** state — a verbatim copy will not boot.

10 Testing

Two Vitest configs, run separately:

	UNIT (VITEST.CONFIG.TS)	INTEGRATION (VITEST.INT.CONFIG.TS)
Files	**/*.test.ts(x) (36 files)	**/*.int.test.ts (6 files)
Environment	node, or jsdom per-file (// @vitest-environment jsdom)	node
Database	none (pure rules + fakes)	real Postgres; globalSetup creates/migrates van_reservation_test
Parallelism	default	serialized (fileParallelism: false)

Tests are **co-located** with source. Pure rules dominate the unit suite; integration tests cover the DB client, migrations, schema CHECKs, the whitelist seed, and service.authenticate() against a live database. jsdom tests that touch motion's useReducedMotion stub matchMedia per file.

11 Local development

```
cp .env.example .env          # then set SESSION_SECRET + a provider mode
pnpm install
pnpm db:up                    # Postgres 17 on :5433 (Docker)
pnpm db:migrate && pnpm db:codegen
pnpm dev                      # http://localhost:3000
```

COMMAND	PURPOSE
pnpm dev / build / start	Next dev / production build / serve
pnpm typecheck	tsc --noEmit
pnpm lint / format	Biome
pnpm test / test:int	Unit / integration tests
pnpm db:migrate:make <name> · db:seed · db:codegen	Schema workflow

12 Email (Amazon SES)

Outbound email mirrors the auth **provider seam** (§5): nothing depends on SES directly, only on a MailTransport interface. Depth in `email-ses.md`.

```
interface MailTransport {  
  send(message: EmailMessage): Promise<Result<SentMessage, EmailFailure>>;  
}
```

- **Transports**, chosen by `EMAIL_MODE` in `select-transport.ts`: **SesTransport** (`ses`, `@aws-sdk/client-sesv2`) and **StubTransport** (`stub`, the default — captures + logs, never sends, and **refuses to run in production**, so a forgotten `EMAIL_MODE=ses` fails loudly, not silently).
- **Failures are typed** so callers separate retryable from permanent: `RATE_LIMITED` / `SERVICE_UNAVAILABLE` (transient) vs. `EMAIL_REJECTED` / `VALIDATION_FAILED` (permanent — don't retry unchanged).
- **Templates** are React Email components rendered to `{ subject, html, text }`; dates are passed **pre-formatted** (an email has no timezone).

Built but **not yet wired**: `sendBookingConfirmation()` is tested and ready, but the booking write-flows that would trigger it don't exist yet.

13 Notable pre-production items

- **Roles are cached in the JWT** (§5.5) — decide on immediate-revocation strategy before admin write operations ship.
- **Middleware** → `proxy.ts`: `middleware` is deprecated in Next 16, but the file is kept deliberately — it runs on the edge, which is what **prevents** the Kysely/pg runtime from being pulled into the route gate. Migrate together with an `import-restriction` lint rule for that file.
- **Fill the roster**: set `ROSTER_ALLOWLIST`, seed real admin identities in `admin_whitelist`, and confirm `ROSTER_ALLOW_ALL_WHEN_EMPTY` is off in prod.
- **Assets & fonts**: license/install the Elevance Sans web font (falls back to Inter), and confirm licensing on the van product photograph.
- **dev-identities.ts** is fixture data for `AUTH_MODE=stub`; it is unused in `cgsauth` mode and should be removed once `stub` mode is retired.

- **SES go-live:** verify a sender identity (+ DKIM), leave the SES sandbox, set EMAIL_MODE=ses + credentials, and wire `sendBookingConfirmation()` to the booking events once those write-flows exist. See `email-ses.md`.